Programación Orientada a Objetos (POO/OOP)
Introducción y conceptos básicos

La **programación orientada a objetos** (OOP) es un **paradigma de programación** que organiza el código en **objetos**. Un objeto es una **entidad** que tiene datos y comportamiento. Los datos de un objeto se almacenan en **atributos** y el comportamiento de un objeto se implementa en **métodos**.

OOP se basa en cuatro principios fundamentales:

- **Abstracción:** La abstracción es el proceso de ocultar los detalles de implementación de un objeto y exponer sólo su interfaz. Esto permite que los usuarios de un objeto se concentren en su comportamiento y no en sus datos internos.
- **Encapsulamiento:** El encapsulamiento es el proceso de agrupar datos y comportamiento juntos en un objeto. Esto ayuda a proteger los datos de un objeto de acceso no autorizado y también ayuda a mantener el código organizado.
- **Herencia:** La herencia es el proceso de crear nuevos objetos a partir de objetos existentes. Esto permite que los nuevos objetos hereden las propiedades y métodos de los objetos existentes.
- **Polimorfismo:** El polimorfismo es la capacidad de un objeto para comportarse de diferentes maneras en diferentes situaciones. Esto se logra utilizando métodos virtuales, que son métodos que pueden ser reemplazados por métodos específicos de la clase hija.

También podemos agregar

- **Modularidad:** Característica que permite dividir una aplicación en varias partes más pequeñas (denominadas módulos), independientes unas de otras.
- **Ocultación (aislamiento):** Los objetos están aislados del exterior, protegiendo a sus propiedades para no ser modificadas por aquellos que no tengan derecho a acceder a las mismas.
- **Recolección de basura:**Es la técnica que consiste en destruir aquellos objetos cuando ya no son necesarios, liberándolos de la memoria.

OOP ofrece una serie de **ventajas** sobre otros paradigmas de programación, como la programación procedural. Estas ventajas incluyen:

- **Mejor diseño:** OOP facilita el diseño de programas más modulares y fáciles de entender.
- **Mejor mantenimiento:** OOP facilita el mantenimiento de programas, ya que los cambios en un objeto solo afectan a los usuarios de ese objeto.
- **Mejor escalabilidad:** OOP facilita la escalabilidad de programas, ya que los nuevos objetos se pueden crear fácilmente a partir de objetos existentes.
- **Mejor reutilización de código:** OOP facilita la reutilización de código, ya que los objetos se pueden reutilizar en diferentes programas.

OOP es un paradigma de programación poderoso que puede ser utilizado para crear programas más eficientes, escalables y fáciles de mantener.

Abstracción

Un ejemplo de abstracción es una clase de objetos. *Una clase es un modelo de un objeto que define sus datos y comportamiento. Los objetos individuales se crean a partir de clases.*

Por ejemplo, una clase llamada `Persona` puede definir los datos `nombre`, `apellido` y `edad`. También puede definir los métodos `getNombre()`, `getApellido()` y `getEdad()`. Los objetos individuales de la clase `Persona` se crearían con diferentes valores para los datos `nombre`, `apellido` y `edad`.

Los usuarios de la clase `Persona` solo necesitarían saber que la clase tiene los datos `nombre`, `apellido` y `edad` y los métodos `getNombre()`, `getApellido()` y `getEdad()`. No necesitarían saber cómo se implementan estos datos y métodos.

La abstracción es una herramienta poderosa que puede ayudar a hacer que el código sea más fácil de entender y usar. Al ocultar los detalles de implementación de un objeto, la abstracción permite que los usuarios de un objeto se concentren en su comportamiento. Esto puede hacer que el código sea más modular, más fácil de mantener y más fácil de reutilizar.

Aquí hay algunos ejemplos de abstracción en programación orientada a objetos:

- **Clases:** Una clase es un modelo de un objeto. Una clase define los atributos y métodos de un objeto.
- **Objetos:** Un objeto es una instancia de una clase. Un objeto tiene los atributos y métodos definidos por su clase.
- **Métodos:** Un método es una función que pertenece a un objeto. Un método puede acceder y modificar los atributos de un objeto.
- **Atributos:** Un atributo es una variable que pertenece a un objeto. Un atributo puede almacenar datos sobre un objeto.

Encapsulamiento

El encapsulamiento se logra utilizando modificadores de acceso. Los **modificadores de acceso** son palabras clave que se utilizan para especificar el nivel de acceso a un miembro de datos o método. Los modificadores de acceso más comunes son:

- **público (public):** Cualquier código puede acceder al miembro de datos o método.
- **privado (private):** Solo el código de la clase en sí puede acceder al miembro de datos o método.
- **protegido (protected):** El código de la clase en sí y el código de las clases derivadas pueden acceder al miembro de datos o método.

Por ejemplo, el siguiente código muestra cómo usar el modificador de acceso privado para proteger un miembro de datos llamado `nombre` de acceso no autorizado:

```
class Persona {
    private string $nombre;

    public function getNombre() {
        return $this->nombre;
    }

    public function setNombre($nombre) {
        $this->nombre = $nombre;
    }
}
```

En el código anterior, el miembro de datos `nombre` se declara como privado. Esto significa que solo el código de la clase `Persona` puede acceder al miembro de datos `nombre`. El código fuera de la clase `Persona` no puede acceder al miembro de datos `nombre`.

El encapsulamiento es una herramienta poderosa que puede ayudar a hacer que el código sea más seguro y más fácil de mantener. Al proteger los datos de un objeto de acceso no autorizado, el encapsulamiento puede ayudar a evitar errores y fallos en el programa.

Aquí hay algunos otros ejemplos de encapsulamiento en programación orientada a objetos:

- Usar modificadores de acceso para proteger miembros de datos de acceso no autorizado.
- Usar métodos getter y setter para acceder y modificar datos.
- Usar interfaces para exponer solo la funcionalidad pública de un objeto.

Herencia

La herencia se puede usar para crear una jerarquía de objetos. En una jerarquía de objetos, una clase se puede basar en otra clase. La clase base es la clase de la que se deriva la clase hija. La clase hija hereda las propiedades y métodos de la clase base.

La herencia se puede usar para crear diferentes tipos de objetos de un conjunto común de propiedades y métodos. Por ejemplo, se puede tener una clase `Persona` que tenga propiedades como `nombre`, `apellido` y `edad`. Se puede tener una clase `Estudiante` que se base en la clase `Persona` y tenga propiedades adicionales como `matrícula` y `carrera`.

La herencia se puede usar para reutilizar código. Si tiene una clase que se usa en varios lugares, puede crear una clase base y hacer que las otras clases se basen en la clase base. Esto le permitirá realizar cambios en la clase base y que los cambios se apliquen automáticamente a todas las otras clases.

La herencia es una herramienta poderosa que puede ayudar a hacer que el código sea más fácil de escribir, mantener y reutilizar.

Aquí hay algunos ejemplos de herencia en programación orientada a objetos:

- Una clase `Animal` puede ser la clase base de clases específicas de animales, como `Perro` y `Gato`. La clase `Animal` puede tener propiedades comunes a todos los animales, como `nombre` y `edad`, mientras que las clases `Perro` y `Gato` pueden tener propiedades específicas a su especie, como `raza` y `pelaje`.
- Una clase `Vehículo` puede ser la clase base de clases específicas de vehículos, como `Automóvil` y `Camión`. La clase `Vehículo` puede tener propiedades comunes a todos los vehículos, como `marca` y `modelo`, mientras que las clases `Automóvil` y `Camión` pueden tener propiedades específicas a su tipo, como `número de puertas` y `capacidad de carga`.

Para crear una nueva clase heredada, se utiliza la palabra clave `extends`. Por ejemplo, el siguiente código crea una nueva clase llamada `Vehiculo` que hereda de la clase `Transporte`:

```
class Vehiculo extends Transporte {

}
```

Polimorfismo

El polimorfismo se puede usar para hacer que el código sea más flexible y más fácil de mantener. Por ejemplo, si tiene una clase `Animal` con un método `hacerSonido()`, puede tener diferentes clases que deriven de la clase `Animal`, como `Perro` y `Gato`. La clase `Perro` puede implementar el método `hacerSonido()` para que haga que el perro ladre, mientras que la clase `Gato` puede implementar el método `hacerSonido()` para que haga que el gato maulle.

De esta manera, el código que llama al método `hacerSonido()` no necesita saber qué tipo de animal está llamando al método. El código solo necesita saber que el objeto es un objeto `Animal`. El polimorfismo se encargará de llamar a la implementación correcta del método `hacerSonido()`, dependiendo del tipo de objeto que se esté llamando.

Aquí hay algunos ejemplos de polimorfismo en programación orientada a objetos:

- Usar métodos virtuales para permitir que diferentes objetos respondan de manera diferente a la misma llamada.
- Usar interfaces para especificar la interfaz de un objeto, pero dejar que la implementación sea específica de la clase.
- Usar clases abstractas para crear una clase que no se puede instanciar, pero que se puede usar como base para otras clases.

¿Qué es una clase y un objeto?

En programación, una **clase** es un modelo de un **objeto**. Define los datos y el comportamiento del objeto. Los objetos individuales se crean a partir de clases.

Una clase se define utilizando una declaración de clase. La declaración de clase incluye el nombre de la clase, las propiedades de la clase y los métodos de la clase.

Las propiedades son variables que pertenecen a una clase. Los métodos son funciones que pertenecen a una clase.

Una instancia de una clase es un objeto que se crea a partir de la clase. Las instancias de la clase comparten las propiedades y los métodos de la clase.

Las clases se utilizan para crear objetos que son similares, pero que pueden tener algunas diferencias. Por ejemplo, puede tener una clase `Person` que tiene las propiedades `nombre` y `edad`. Puede tener diferentes instancias de la clase `Person` con diferentes nombres y edades.

Las clases se utilizan para organizar el código y hacerlo más fácil de leer y mantener.

Aquí hay algunos ejemplos de clases en PHP:

```php
class Person {
    public $name;
    public $age;


    public function __construct($name, $age) {
        $this->name = $name;
        $this->age = $age;
    }


    public function sayHello() {
        return "Hello, my name is $this->name. I am $this->age years old.";
    }
}


class Car {
    public $make;
    public $model;
    public $year;


    public function __construct($make, $model, $year) {
        $this->make = $make;
        $this->model = $model;
        $this->year = $year;
    }


    public function drive() {
        return "The car is driving.";
    }
}
```

Estas clases se pueden utilizar para crear objetos de personas y coches. Por ejemplo, el siguiente código creará un objeto de persona llamado `john doe` que tiene 30 años de edad:

```
$person = new Person("John Doe", 30);
```

El siguiente código llamará al método `sayHello()` del objeto `person` y devolverá el siguiente texto:

```
echo $person->sayHello(); // Hello, my name is John Doe. I am 30 years old.
```

El siguiente código creará un objeto de coche llamado toyota prius que fue fabricado en 2010:

```
$car = new Car("Toyota", "Prius", 2010);
```

El siguiente código llamará al método drive() del objeto car y devolverá el siguiente texto:

```
echo $car->drive(); //The car is driving.
```